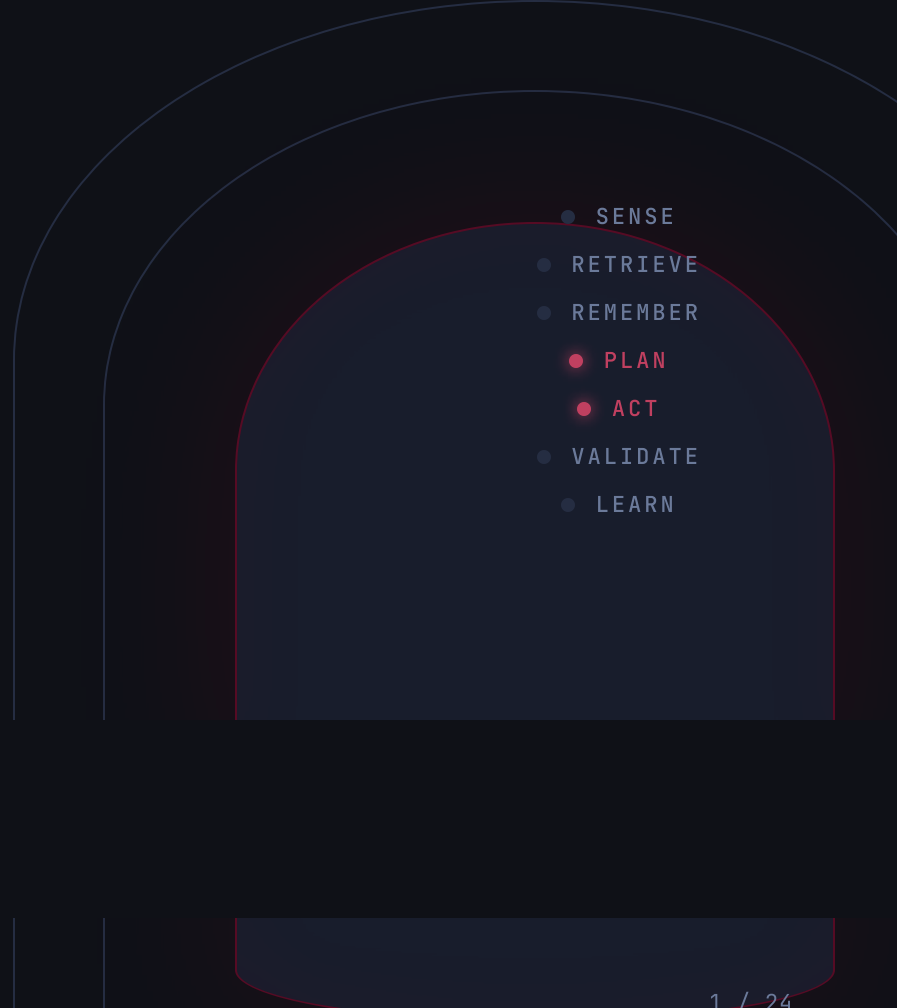


The model is the engine. The **harness** is the car.

What makes a good agentic coding harness — and how clew keeps it coherent.

- 
- A diagram illustrating the agentic coding harness cycle. It features three concentric, rounded rectangular shapes. The innermost shape is dark blue and contains a vertical list of seven items, each preceded by a small circle. The second shape is a lighter blue and is positioned around the first. The outermost shape is a very light blue and is positioned around the second. The list of items is: SENSE, RETRIEVE, REMEMBER, PLAN, ACT, VALIDATE, and LEARN. The items PLAN and ACT are highlighted with red circles, while the others have grey circles.
- SENSE
 - RETRIEVE
 - REMEMBER
 - PLAN
 - ACT
 - VALIDATE
 - LEARN

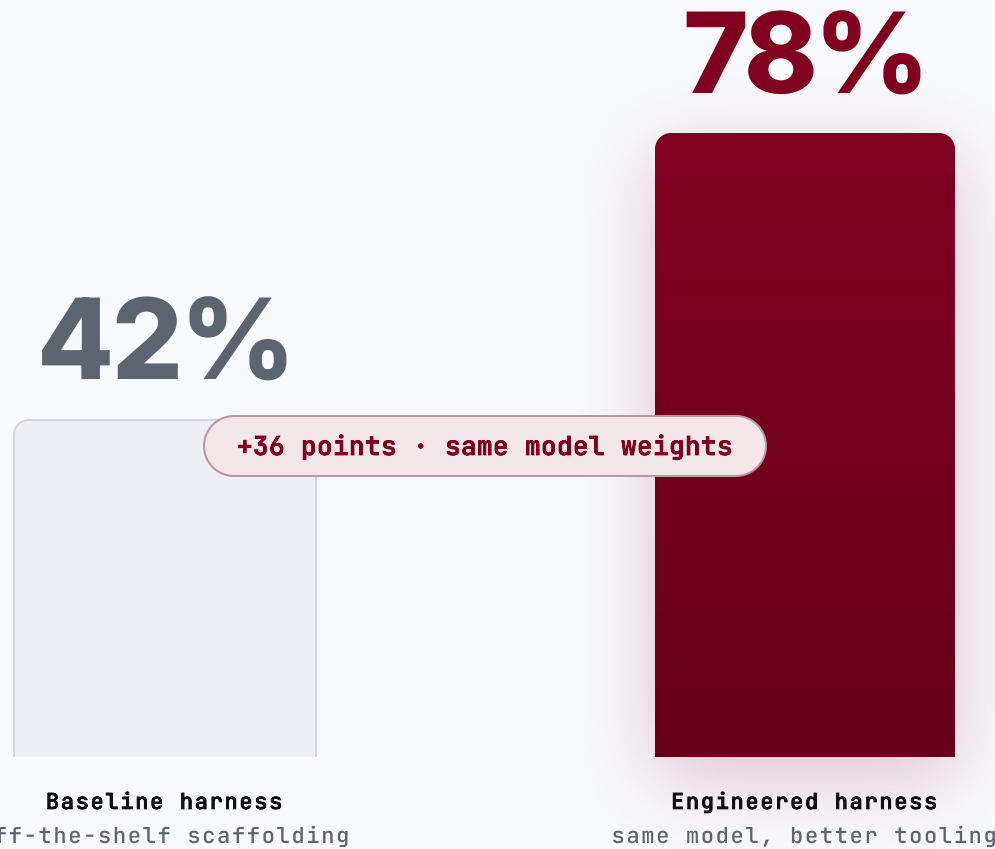
SECTION 01 / 04

The Agentic **Harness**

Why the tooling around the model — not the model — is the performance lever.

01

Hold the model fixed, swap the harness: 42% → 78%.



Claude Opus 4.5 • HAL CORE-Bench • the only thing that changed was the harness

Performance lives in the tooling around the model — not the model.

Ablation studies rank what actually moves agent performance. Model prompt-tuning sits **last**.

1 Tools



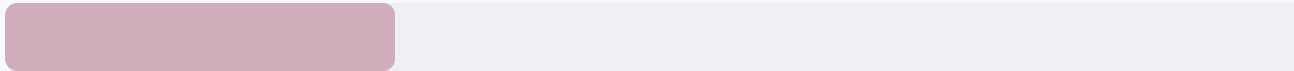
2 Middleware



3 Long-term memory



4 System prompt



THE INVERSION

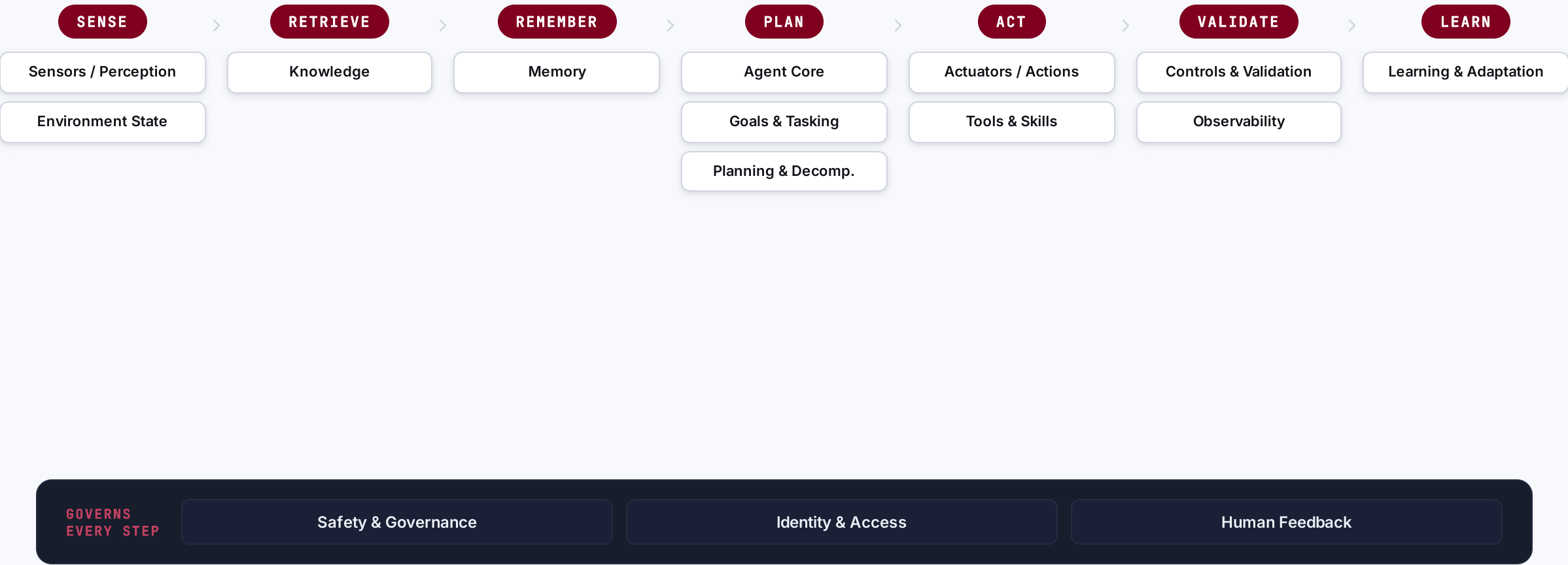
Most teams pour effort into #4 — the prompt. The gains sit in #1-3.

THE MODEL

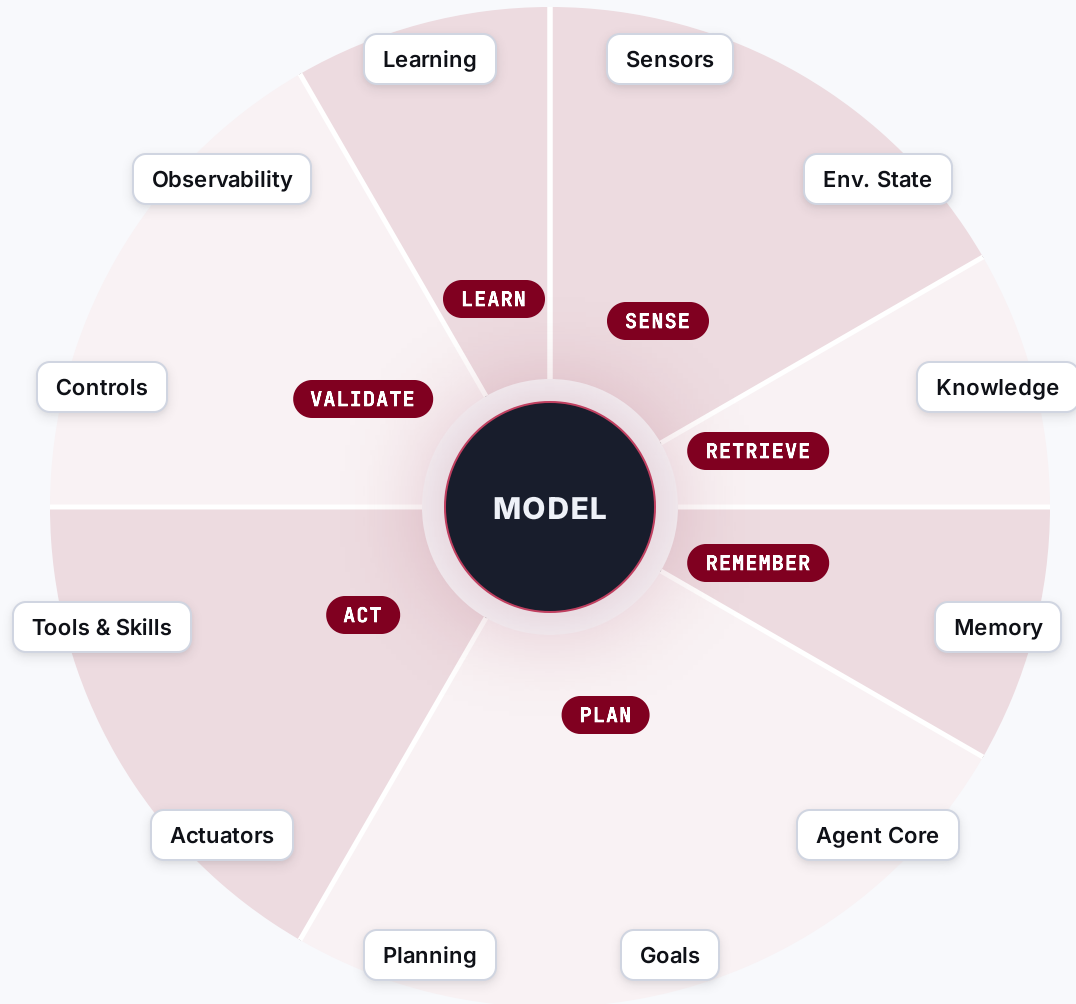
Table stakes once it is capable. The harness is the differentiator.

One loop, 15 capabilities — a capability system, not a layer cake.

The model fills each step; the harness governs the transitions — and three capabilities govern them all.



The model is a dot. The harness is the mass around it.



CROSS-CUTTING

Govern every step — they don't belong to one.

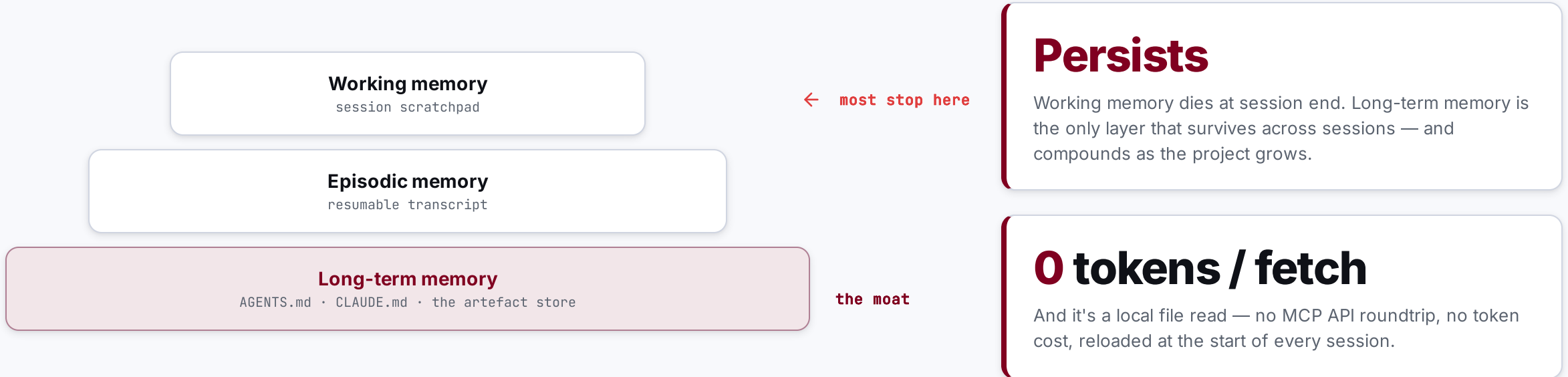
✓ Safety & Governance

🔑 Identity & Access

👤 Human Feedback

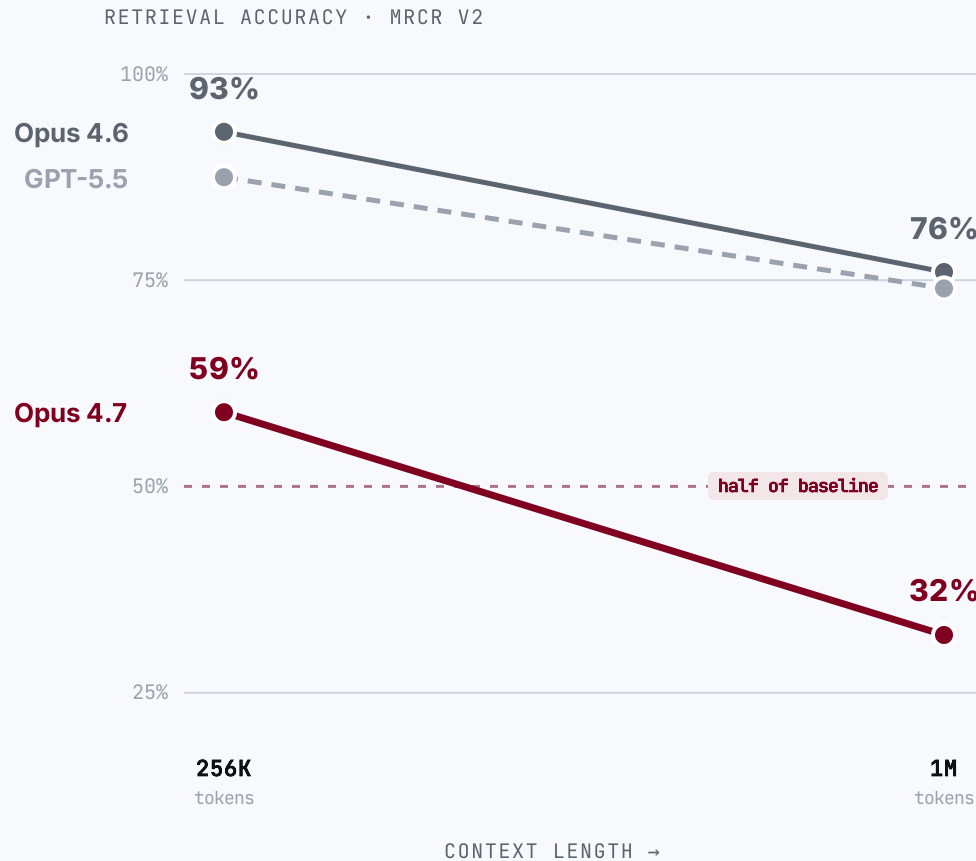
Each wedge is **one loop step** and the capabilities it owns · the three on the right **govern every step**.

Most harnesses only have the shallowest memory — yet memory is the moat.



A bigger context window is not bigger memory.

Retrieval accuracy **rots** as you fill the window — even the 2026 frontier.



The advertised window is **not the usable window.**

59% → 32%

Claude Opus 4.7 loses ~half its multi-needle retrieval from 256K to 1M tokens. Opus 4.6 holds better (93% → 76%).

Codex too

the GPT-5.x line behind Codex CLI drops ~87% → 74% at long context — a proxy; no per-length figure is published for the Codex-tuned model.

11 of 13

models fell below half their accuracy by 32K tokens — context rot isn't new (NoLiMa, 2025).

SECTION 02 / 04

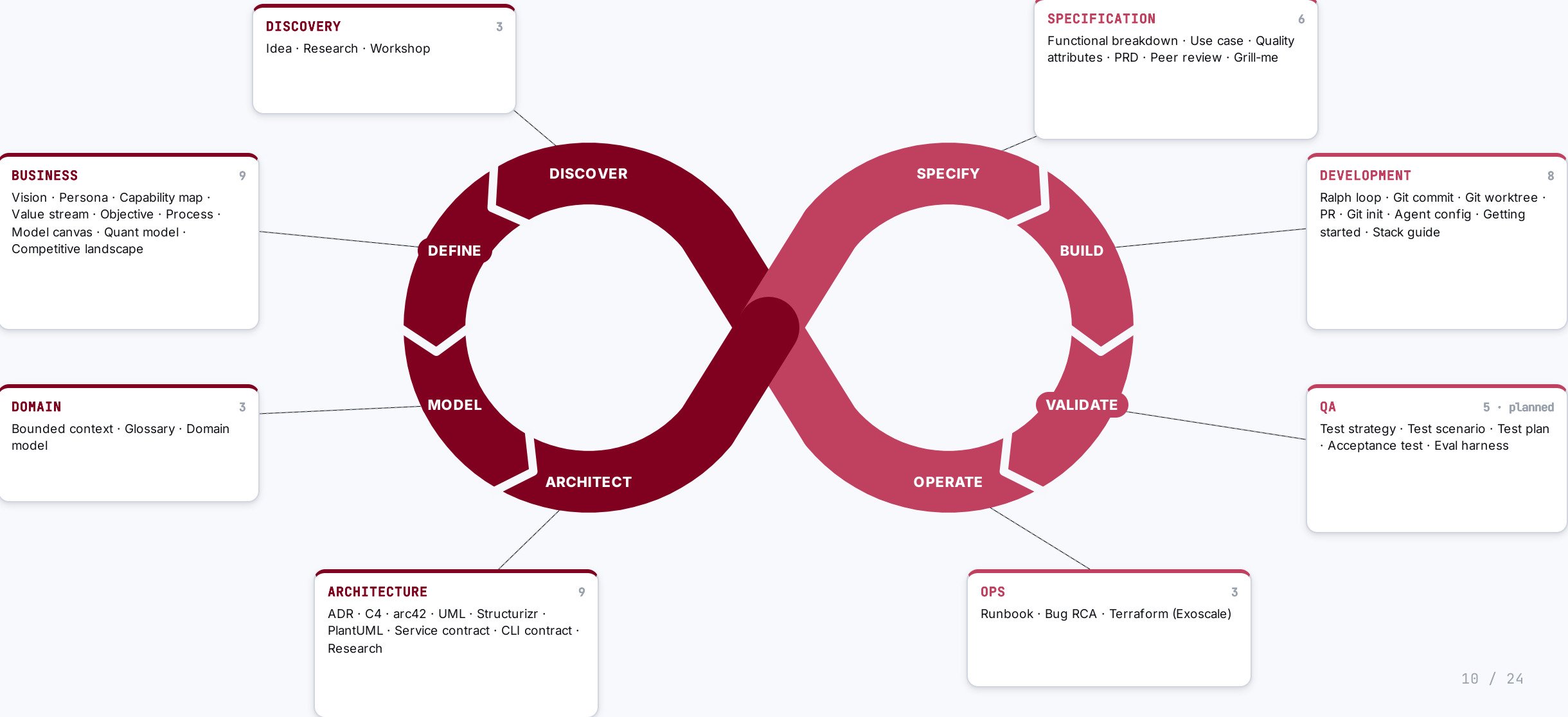
The Metamodel

How an agent armed with skills and rules turns business context into deployable artefacts — every step traceable.

02

Skills & rules wrap the whole lifecycle — from discovery to operations.

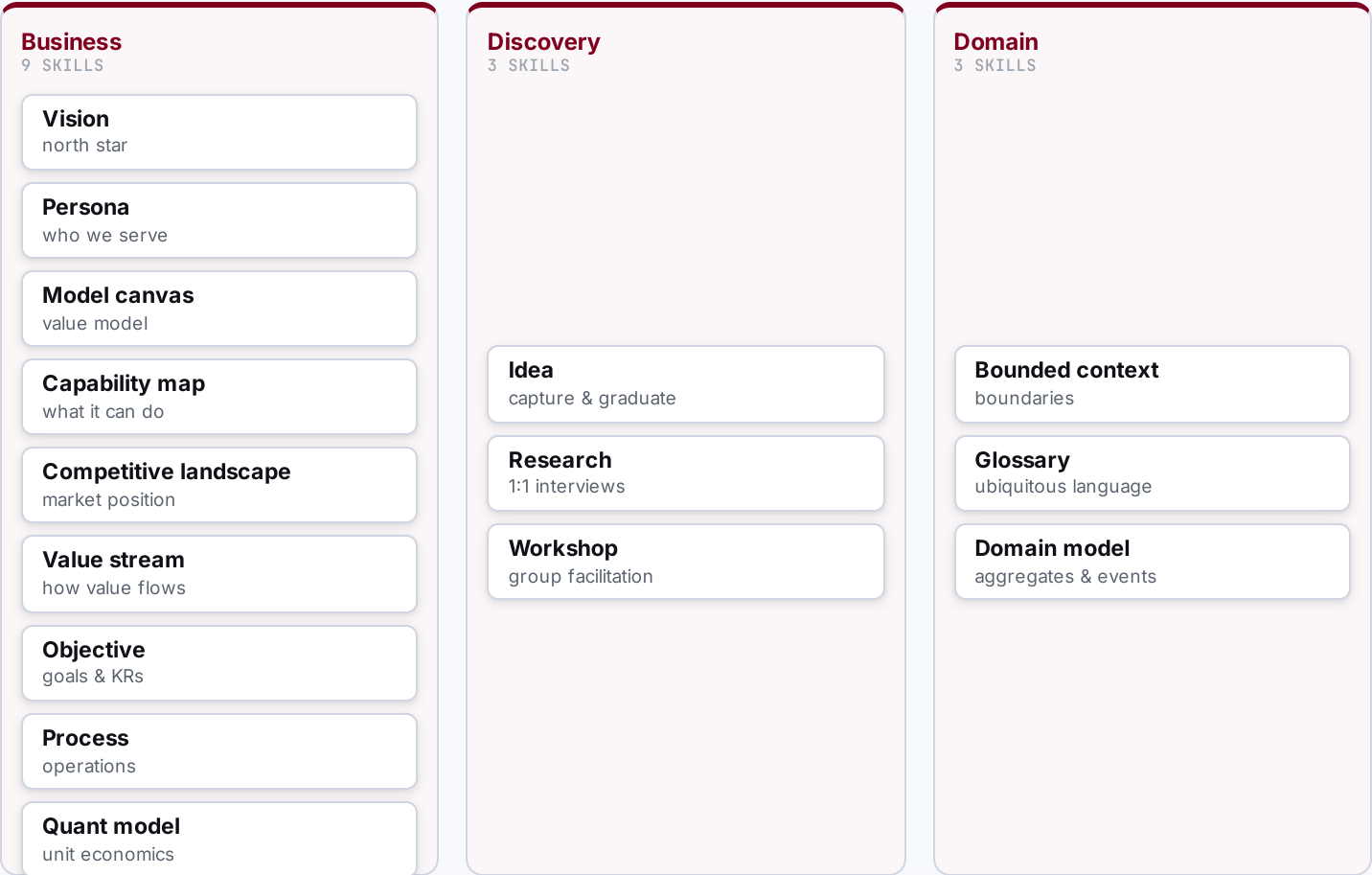
Far beyond DevOps: eight phases from business discovery to operations, each owned by a prefix, all tracing back to one metamodel.



Discover & Define — make the context real.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Brainstorm & capture**
ideas, hypotheses
- **Research & scan**
market, competitors
- **Interview & workshop**
with the business
- **Model the business**
vision → objectives
- **Define the domain**
ubiquitous language



Architect the solution — turn context into a solution.

WHAT THE AGENTIC ENGINEER DOES HERE

- 

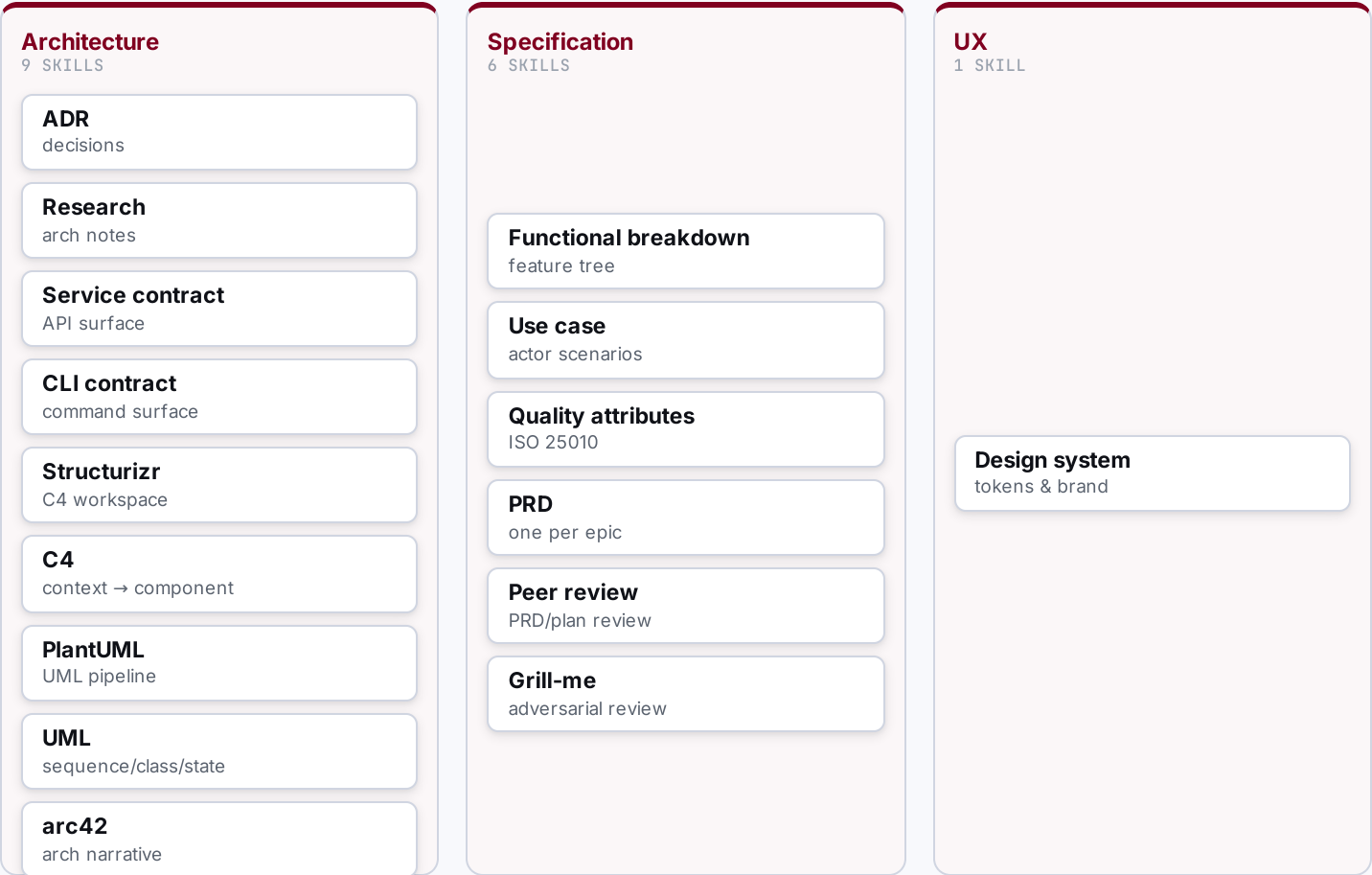
Decide architecture
ADRs, C4, arc42
- 

Define contracts
service, CLI
- 

Break down function
FBS
- 

Specify & qualify
use cases, quality
- 

Write & review PRDs
one per epic



Execute & Deliver — plan, build, test, ship.

WHAT THE AGENTIC ENGINEER DOES HERE

- 

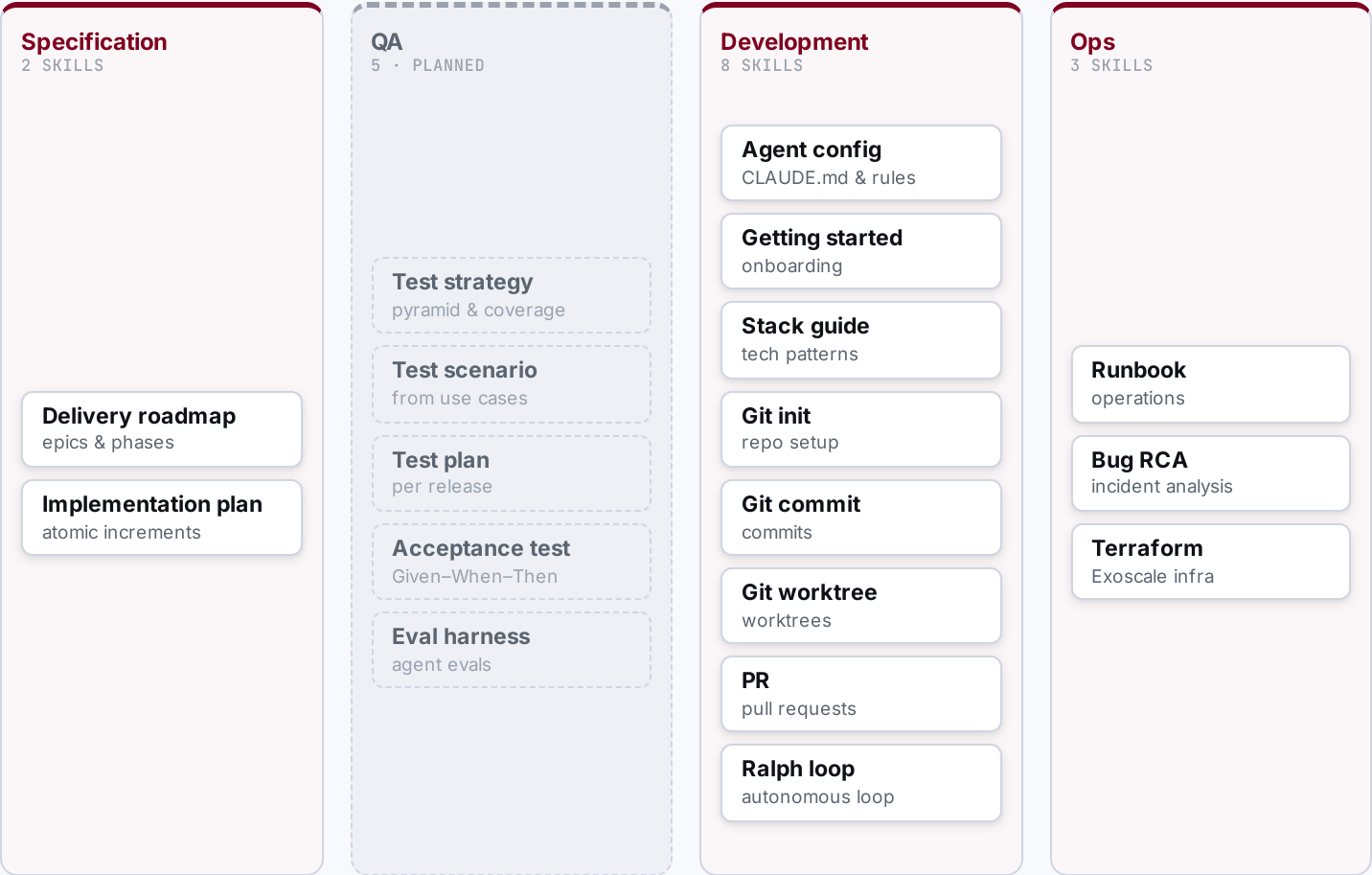
Sequence the roadmap
epics, phases
- 

Plan increments
atomic steps
- 

Build & validate
tests, checks
- 

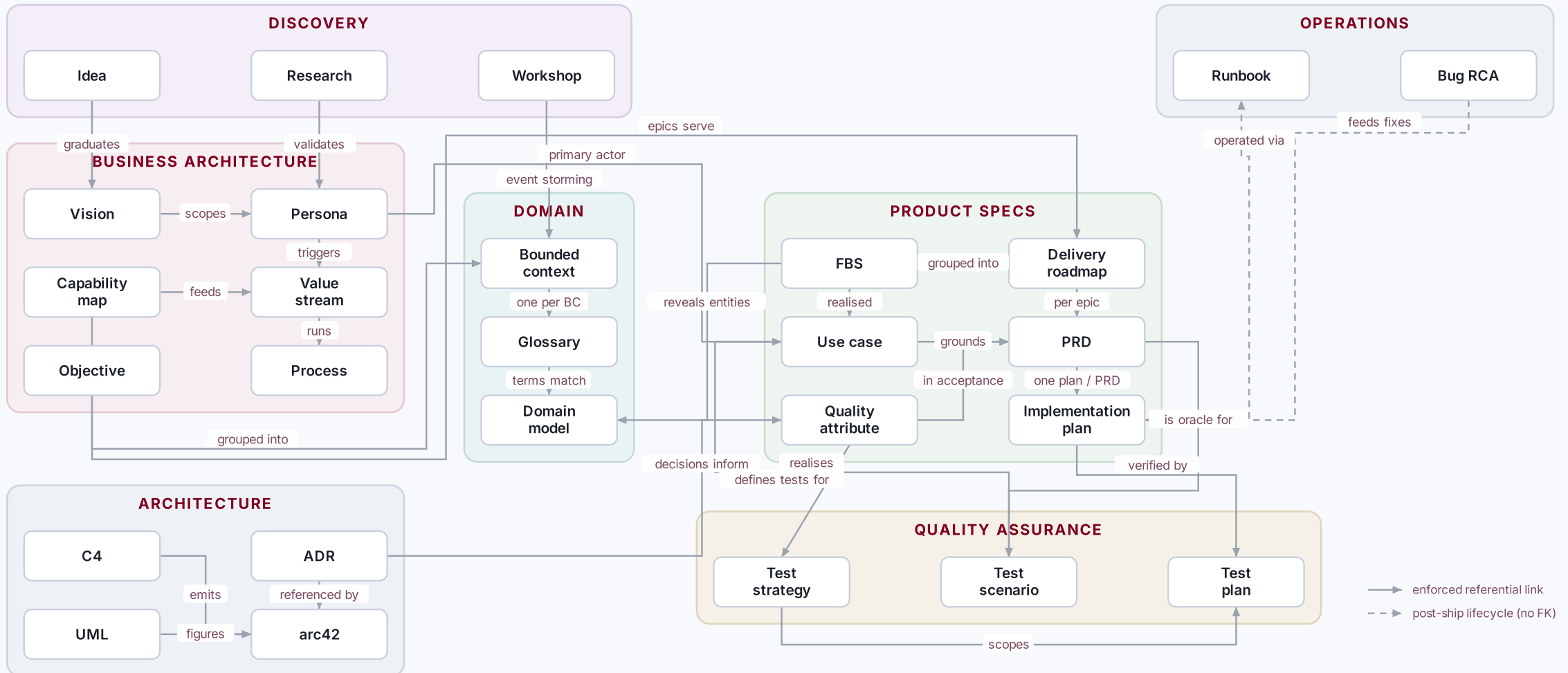
Deploy
infra, release
- 

Operate & fix
runbooks, RCA



Everything is related — the metamodel as one graph.

Seven domains, twenty-seven artefacts; solid edges are referential links the engine enforces at write time — dashed are loose post-ship lifecycle.



SECTION 03 / 04

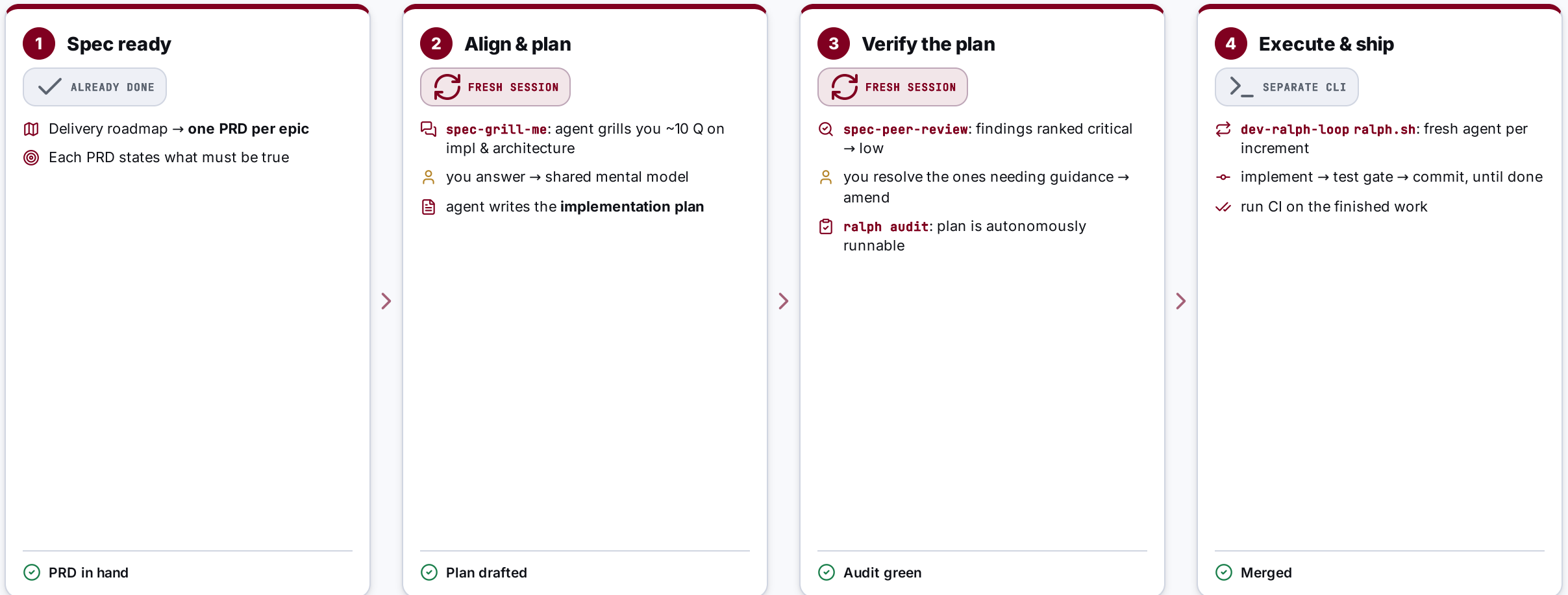
The Build Workflow

From a PRD to merged code — how one engineer drives agents through alignment and verification gates, then lets them run autonomously.

03

From PRD to merged code — two fresh-session gates, then autonomy.

Business context and discovery are done; the roadmap gives one PRD per epic. The loop below runs per PRD.



Two **fresh-session** checks (grill-me before, peer-review after) keep me and the agent aligned; the loop only runs once the plan is proven autonomous.

PRD says what must be true; the plan says how — safely.

Two artefacts, one chain: the PRD scopes the epic, the implementation plan breaks it into runnable increments.

BUILD BY FEATURE · ONE PER EPIC

PRD

“What must be true, and why.”

-  **Problem, goals & non-goals** — the scope boundary
-  **User stories** grounded in personas (P-NN)
-  **Acceptance criteria** + success metrics
-  **\$0 traceability** — the epic and its dependencies

E-NN · P-NN · C-N.M.FXX · QA-XXNN · UC-NN

1 PRD
↓
1 plan



acceptance criteria
become
the plan’s test gates

increments
deliver
the PRD scope

ATOMIC INCREMENTS · ONE PER PRD

Implementation plan

“How we get there, reversibly.”

-  **Ordered increments** (Inc-1 ... Inc-N)
-  each **small · testable · reversible**
-  scope + primary files per increment
-  **test gate** + exit criteria = done

Plan-NNNN · Inc-N · Status: pending → done

Two fresh agents pressure-test the work — before and after the plan.

Grill-me pulls the thinking out of my head; peer-review checks the document. Both run in a clean session.

ALIGNMENT · BEFORE THE PLAN

spec-grill-me

Live Socratic session on the PRD — one question at a time.

- 🔍 Agent asks ~10 focused Q on **implementation & architecture**
- 📖 Vocabulary checked against the **domain glossary**
- 📝 Decisions crystallised into **ADRs** while context is live
- 🧠 Surfaces the **thinking gaps** only questioning reveals

Output: **shared mental model** → the agent then writes the implementation plan.

VERIFICATION · AFTER THE PLAN

spec-peer-review

Static, independent scan of the finished plan.

- 🔍 Reads the plan end-to-end, builds a requirement map
- ⚠️ Findings ranked with concrete remediation:

critical major normal low
- 🔧 I resolve the ones needing guidance → **amend the plan**

Output: **validated plan** → ready for the autonomy audit.

The Ralph loop — a fresh agent per increment, until the plan is done.

WHAT THE LOOP IS

- ↻ A bash script (**ralph.sh**) drives the plan to completion — one increment per iteration.
- ↻ **Fresh agent each iteration** → clean context, no drift or context rot.
- 🚩 Loops while increments are **pending**; stops when every one is **Status: done**.

```
ralph.sh <workspace> \  
  --max-iterations 50 --with-prd --with-push  
# spawns: claude --print < iteration-prompt
```

🔄 ONE ITERATION · PERCEIVE → IMPLEMENT → VALIDATE → COMMIT

Audit must be green before the first run.



Pick next pending increment

read workspace + plan



Implement its scope

follow the primary-files list



Run the test gate

pottery wheel — up to 3 retries



Commit the increment

advance status → done

↑ repeat with a fresh agent



All increments done → run CI

SECTION 04 / 04

clew

The engine that persists the whole graph: deterministic IDs, write-time integrity, and end-to-end traceability.

04

clew engineers the deepest memory layer — so it stops rotting.

WITHOUT STRUCTURE, MEMORY ROTS



No traceability graph from the docs

No reliable view of how artefacts relate — joins across them are impossible.



No determinism if you trust the LLM

It silently rennumbers IDs; you end up re-verifying everything by hand.



Refactors cost a fortune

Change one ID or name and the effort to update every reference is massive.

CLEW = STRUCTURED LONG-TERM MEMORY



End-to-end traceability

a typed graph generated from the docs themselves



Write-time integrity

deterministic IDs • FK checks • drift detection



Zero token cost per fetch

local file reads • the repo is the source of truth

IDs minted by the app, links checked at write time — never by the LLM.

1 • AGENT ASKS

Call clew via Bash

```
clew new prd --epic E-01
```

The agent never invents an ID — it requests one.

2 • CLEW ENFORCES

Validate & mint

```
E-01 exists? ✓  
FK enforced · → PRD-0001
```

Type-checked against the relationship registry;
collisions rejected.

3 • AGENT WRITES

Reference the ID

```
[PRD-0001](prd-0001.md)
```

Markdown prose cites the stable ID clew returned.



Deterministic — clew mints every ID and checks every link



Guessed — the LLM rennumbers silently and references rot

Ask the graph anything: trace upstream, measure blast radius, prove integrity.

clew trace OBJ-02

OBJ-02
↑ E-01 Epic
↑ C1.2 Capability
↑ P-01 Persona
↑ VISION

Upstream chain — why this exists.

clew impact E-03

E-03 changes →
↓ PRD-0007
↓ QA-SC02
↓ Plan-0007
3 artefacts affected

Downstream blast radius — what breaks.

clew check

scanning references...
bindings ✓
drift ✓
orphans ✓
0 broken references

Integrity audit — deterministic, repeatable.

100% current Every internal reference is up to date, **all the time** — the north-star metric is end-to-end traceability, not work-in-progress.

THE TAKEAWAY

Engineer the **harness**, not just the prompt.

Start with the highest-leverage capability: memory that doesn't rot.

Not a PM tool

Not a wiki

Not a SaaS

The repo is the source of truth

Git for product architecture — manage what must be true, not what's in progress.